G. PULLAIAH COLLEGE OF ENGINEERING AND TECHNOLOGY
Autonomous · Affiliated to JNTUA, Anantapur · Kurnool

A40531 · DEEP LEARNING · IV B.Tech CSE · I Semester

UNIT I

Model Answers for the 10-Mark Question Bank

Derivations · formulas · diagrams · examination keywords

Linear Algebra, Probability and Numerical Computation

Q1 PCA procedure using covariance and eigen-decomposition

Q2 Eigen-decomposition versus SVD as routes to PCA

Q3 Overflow and underflow with numerical safeguards

Q4 Expectation, variance, covariance and the motivation for PCA

Q5 Gradient-descent derivation and the effect of the learning rate

Q6 Constrained optimisation with a norm constraint and regularisation

Q7 Least squares: normal equations and uniqueness

Q8 PCA before a neural network with correlated features

Every answer is written to the 10-mark pattern: definition, derivation, formula, diagram, comparison table and conclusion, followed by the keywords that examiners look for.

08
MODEL ANSWERS

57
NUMBERED FORMULAE

18
VECTOR DIAGRAMS

Contents

Unit I — Linear Algebra, Probability and Numerical Computation

Q1. PCA procedure 3

Q2. Eigen vs SVD 7

Q3. Overflow and underflow 9

Q4. Expectation, variance, covariance 12

Q5. Gradient descent 15

Q6. Constrained optimisation and regularisation 17

Q7. Least squares 20

Q8. PCA before a neural network 22

The unit at a glance

Q	Core idea	Result to memorise
1	Covariance → eigen-decomposition → projection	$C = W\Lambda W^\top$, $Z = XW_k$
2	SVD reaches the same subspace without forming C	$X = U\Sigma V^\top$, $\lambda_i = \sigma_i^2/(m-1)$
3	Never exponentiate large numbers; shift by the maximum	$\log \Sigma_j e^{z_j} = a + \log \Sigma_j e^{z_j - a}$
4	Moments summarise the cloud; PCA diagonalises C	$C = 1/(m-1)X^\top X$, $\mathrm{trace}(C) = \Sigma_i \lambda_i$
5	Steepest descent with a curvature-limited step	$\theta \leftarrow \theta - \eta \nabla J$, $\eta < 2/\lambda_{max}$
6	A norm constraint is a penalty: KKT ⇔ regulariser	$\min J$ s.t. $\|\theta\| \le c \Leftrightarrow \min J + \lambda\|\theta\|^2$
7	Least squares = orthogonal projection	$X^\top Xw = X^\top y$
8	PCA decorrelates and conditions the network's input	$\mathrm{Cov}(Z) = W_k^\top C W_k = \Lambda_k$

HOW TO USE THIS BOOKLET

Each answer follows the same skeleton, so you can revise the *pattern* once and then recall the content: **idea → derivation → formula → diagram → table → conclusion**.

Formula boxes carry equation numbers such as (3.2) — that is *Question 3, second equation*. Figure numbers follow the same rule, so (Fig. 5.1) belongs to Q5.

The **Golden line** is the one sentence to write at the end of an answer; the **Keywords** block is the list an examiner scans for. Aim to reproduce the boxed formulae exactly and to reproduce the diagrams approximately.

Q1 **Given a centered data matrix X, develop a complete PCA procedure using covariance / eigen-decomposition. Explain how the principal components are selected and how dimensionality is reduced.**

CO1 · L5 (Design) · 10 Marks

Aim and notation

Principal Component Analysis (PCA) is an **unsupervised linear transformation** that maps data of dimension n into a smaller number k of new, uncorrelated axes called **principal components**. Let

$$X \in \mathbb{R}^{m \times n}, \quad m = \text{number of samples}, n = \text{number of features} \tag{1.1}$$

where each row $x^{(i)}$ is one sample and each column is one feature. PCA answers a single optimisation question: *which direction w makes the projected data Xw spread out the most?* Maximum spread means maximum retained information.

Step 1 — Centre the data

The question already supplies a **centred** matrix, so the feature means are zero: $\bar{x}_j = 0$. If raw data were given, we must subtract the mean first:

$$X \leftarrow X - \mathbf{1}\mu^\top, \qquad \mu = \frac{1}{m}\sum_{i=1}^{m} x^{(i)} \tag{1.2}$$

Why centring is compulsory: PCA maximises variance. Variance is measured about the mean, and $X^\top X$ equals the scatter about the origin only when the mean is zero. With uncentred data the first component simply points towards the mean vector, which carries no information about spread.

Step 2 — Build the covariance matrix

$$C = \frac{1}{m-1} X^\top X \in \mathbb{R}^{n \times n} \tag{1.3}$$

The diagonal entry C_{jj} is the variance of feature j; the off-diagonal entry C_{jk} is the covariance between features j and k. Hence C encodes both spread and redundancy. The divisor $m-1$ gives the unbiased estimator; with large m the difference from m is negligible.

Properties of C: square $(n \times n)$, real, **symmetric** $(C^\top = C)$, and **positive semi-definite** $(u^\top C u = 1/(m-1)\|Xu\|^2 \geq 0$ for every $u)$. Symmetry guarantees real eigenvalues and mutually orthogonal eigenvectors; positive semi-definiteness guarantees $\lambda_i \geq 0$.

Step 3 — Eigen-decomposition of the covariance matrix

$$C = W \Lambda W^\top, \qquad W = [w_1, w_2, \ldots, w_n], \quad \Lambda = \text{diag}(\lambda_1, \ldots, \lambda_n) \tag{1.4}$$

Because C is symmetric, W is **orthonormal** ($W^\top W = I$, so $W^{-1} = W^\top$) and the λ_i are real and non-negative. The eigenvectors are the candidate principal directions and the eigenvalues are the variances along them.

Step 4 — Why the eigenvectors maximise variance (derivation)

For a unit direction w the variance of the projected data is the Rayleigh quotient:

$$\text{Var}(Xw) = \frac{1}{m-1} w^\top X^\top X w = w^\top C w \tag{1.5}$$

We therefore solve the **constrained maximisation**

$$\max_{w} w^\top C w \quad \text{subjectto} \quad w^\top w = 1 \tag{1.6}$$

Form the Lagrangian $L(w, \lambda) = w^\top C w - \lambda(w^\top w - 1)$ and set the gradient to zero:

$$\frac{\partial L}{\partial w} = 2Cw - 2\lambda w = 0 \Longrightarrow Cw = \lambda w \tag{1.7}$$

So the stationary directions are exactly the eigenvectors, and at such a point the achieved variance is

$$w^\top Cw = w^\top(\lambda w) = \lambda\,(w^\top w) = \lambda \tag{1.8}$$

Conclusion: the largest eigenvalue gives the maximum variance, so *eigenvalue = variance captured by that component*. The second component is found by repeating the maximisation subject to $w_2^\top w_1 = 0$ — hence successive principal components are orthogonal (uncorrelated).

Step 5 — Sort and select the top k components

Arrange $\lambda_1 \geq \lambda_2 \geq \ldots \geq \lambda_n \geq 0$ and reorder the columns of W identically. The importance of component i is measured by the **explained variance ratio**:

$$\mathrm{EVR}_i = \frac{\lambda_i}{\sum_{j=1}^{n}\lambda_j}, \qquad \mathrm{Cum}_k = \frac{\sum_{i=1}^{k}\lambda_i}{\sum_{i=1}^{n}\lambda_i} \tag{1.9}$$

Because $\Sigma_i \lambda_i = \mathrm{trace}(C)$ is the **total variance** of the data, Cum_k is the fraction of total information retained. The dimension k is chosen by any of the following standard rules:

- **Cumulative variance rule (most common):** smallest k with $\mathrm{Cum}_k \geq 0.95$ (or 0.99).
- **Scree plot / elbow rule:** plot λ_i against i and cut where the curve flattens.
- **Kaiser rule (standardised data only):** keep components with $\lambda_i > 1$, i.e. above average variance.
- **Task-driven rule:** choose k by cross-validation on the final classifier, not by variance alone.

The retained matrix is

$$W_k = [w_1, w_2, \ldots, w_k] \in \mathbb{R}^{n\times k}, \qquad k < n \tag{1.10}$$

Step 6 — Project the data (dimensionality reduction)

$$Z = XW_k \in \mathbb{R}^{m\times k}, \qquad z^{(i)} = W_k^\top x^{(i)} \tag{1.11}$$

Each sample of n features becomes a vector of k **principal component scores**. The dimension of the representation drops from n to k while the dominant variance is preserved. The transformation is a rotation followed by a drop of coordinates — no information in the retained subspace is destroyed.

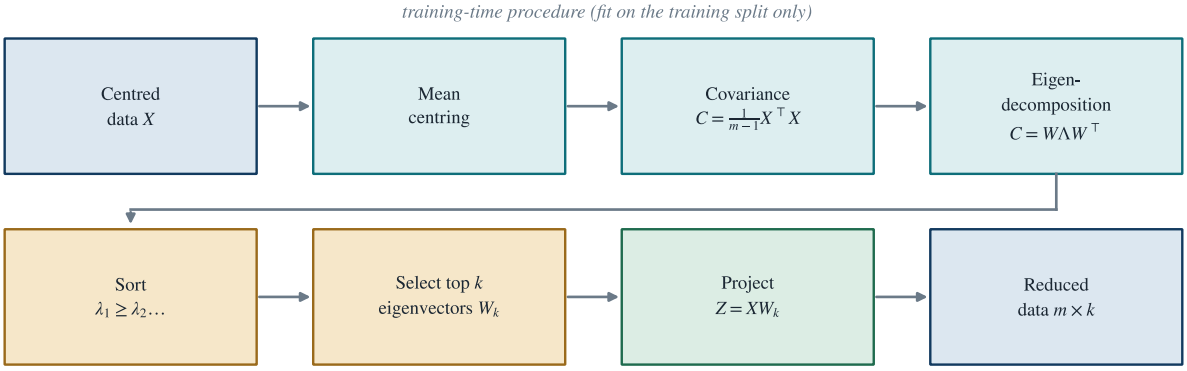


Fig. 1.1 Complete PCA chain: fitting the transform (top row) and the wrap-around continuation to the reduced representation (bottom row).

Step 7 — Properties of the reduced representation

$$\mathrm{Cov}(Z) = W_k^\top CW_k = \Lambda_k = \mathrm{diag}(\lambda_1, \ldots, \lambda_k) \tag{1.12}$$

The covariance of the scores is **diagonal**, so the new features are uncorrelated: PCA simultaneously performs dimensionality reduction **and** decorrelation. Total variance is unchanged by the rotation, $\Sigma_{i=1}^{n}\lambda_i = \text{trace}(C)$, and the first k components carry $\Sigma_{i=1}^{k}\lambda_i$ of it.

Step 8 — Reconstruction and the error it causes

Discarded components can be dropped, but they can also be put back approximately:

$$\hat{x} = W_k z = W_k W_k^{\top} x \approx x, \qquad \|X - \hat{X}\|_F^2 = \sum_{i=k+1}^{n} \lambda_i \tag{1.13}$$

This is the **Eckart–Young theorem**: among all rank-k approximations, the PCA truncation has the smallest squared reconstruction error, and that error is exactly the total variance of the discarded components. It justifies dropping the smallest eigenvalues rather than any other directions.

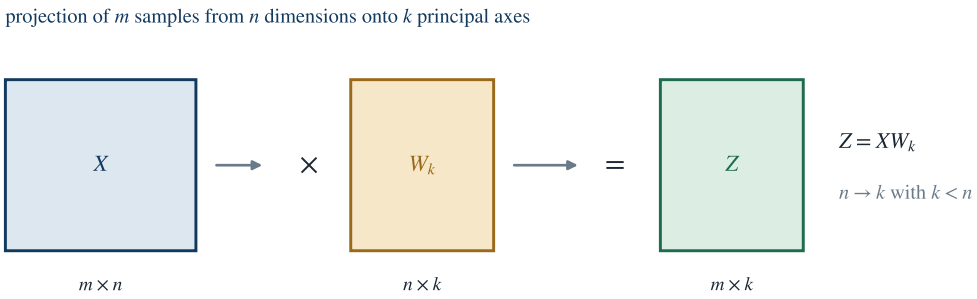


Fig. 1.2 Dimensions of the three matrices involved in the projection step.

Worked numerical example

Take four centred 2-D samples $x^{(1)} = (2, 1)$, $x^{(2)} = (1, 2)$, $x^{(3)} = (-1, -2)$, $x^{(4)} = (-2, -1)$, so $m = 4$, $n = 2$ and the mean is already zero.

Step	Computation	Result
Covariance	$C = X^{\top}X/(m-1)$	diag = variances, off-diag = covariances
Eigenvalues	$\det(C - \lambda I) = 0$	$\lambda_1 = 6, \lambda_2 = 0.667$
Eigenvectors	$Cw = \lambda w$	$w_1 = (0.707, 0.707)^{\top}$, $w_2 = (-0.707, 0.707)^{\top}$
Variance share	$\lambda_i/\Sigma\lambda$	90% and 10%
Projection	$Z = XW_1$	$z_1 = (2.12, 2.12, -2.12, -2.12)$

With $\text{Cum}_1 = 0.90$, the one-dimensional projection keeps 90% of the variance: four samples that needed two numbers each are now described by one number each, and the reconstruction error is $\lambda_2 = 0.667$, exactly the dropped variance.

$$C = \frac{1}{m-1}X^{\top}X = \begin{bmatrix} 3.33 & 2.67 \\ 2.67 & 3.33 \end{bmatrix}, \quad Cw = \lambda w \Rightarrow \begin{bmatrix} 6.00 \\ 0.667 \end{bmatrix} \tag{1.14}$$

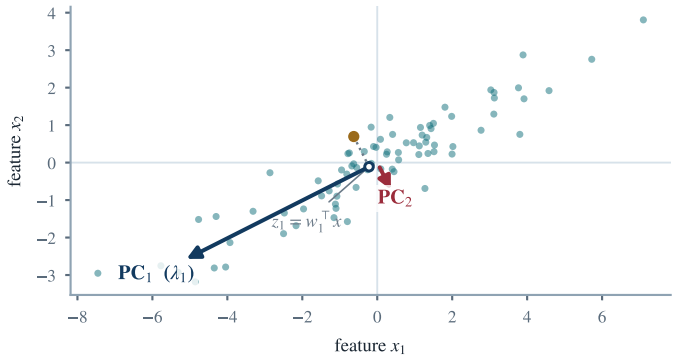


Fig. 1.3 Geometry of the first step: the first principal axis w_1 follows the direction of largest spread, and every point is compressed onto it by $z_1 = w_1^\top x$.

Algorithm (write this in the exam)

1. Centre the data: $X \leftarrow X - \mathbf{1}\mu^\top$.
2. Compute the covariance matrix $C = 1/(m-1)X^\top X$.
3. Eigen-decompose $C = W\Lambda W^\top$.
4. Sort eigenvalues in descending order and reorder W.
5. Choose k from the cumulative explained variance (e.g. $\geq 95\%$).
6. Form W_k from the first k eigenvectors and compute $Z = XW_k$.
7. Optionally reconstruct $\hat{X} = ZW_k^\top$ and report the residual $\Sigma_{i>k}\lambda_i$.

Limitations

- PCA is **linear**; curved manifolds need kernel PCA, autoencoders or t-SNE.
- Components are chosen by **variance, not by relevance** to the target y — a low-variance but highly predictive direction may be discarded.
- PCA is **scale sensitive**: features must be standardised first, otherwise the largest-unit feature dominates the first component.
- Components are linear combinations of all inputs, so interpretability is reduced.
- The eigen route costs $O(n^3)$ (or $O(mn^2)$), which is expensive when n is very large.

EXAMINER'S CHECKLIST

Always state (i) centring, (ii) the covariance formula with $m-1$, (iii) $C = W\Lambda W^\top$, (iv) *eigenvalue = variance*, (v) $Z = XW_k$ with dimensions, and (vi) the cumulative-variance rule for k. Those six items carry most of the ten marks.

GOLDEN LINE

Largest eigenvalue = maximum variance = PC1. Projection $Z = XW_k$ turns n features into k uncorrelated features with minimum loss.

KEYWORDS TO WRITE IN THE EXAM

Principal component · covariance matrix · eigen-decomposition · eigenvalue = variance · orthonormal eigenvectors · explained variance ratio · cumulative variance · $Z = XW_k$ · decorrelation · Eckart–Young · dimensionality reduction

Q2 **Compare eigen-decomposition and SVD as routes to PCA. For a rectangular data matrix, analyse which decomposition is more directly applicable and justify your choice.**

CO1 · L4 (Analyse) · 10 Marks

The two routes

PCA needs only one thing: the orthonormal directions of maximum variance. Those directions can be obtained in two mathematically different ways — from the **square covariance matrix** (eigen-decomposition) or from the **data matrix itself** (singular value decomposition). Both end at the same subspace; they differ in what they factorise, in numerical behaviour and in cost.

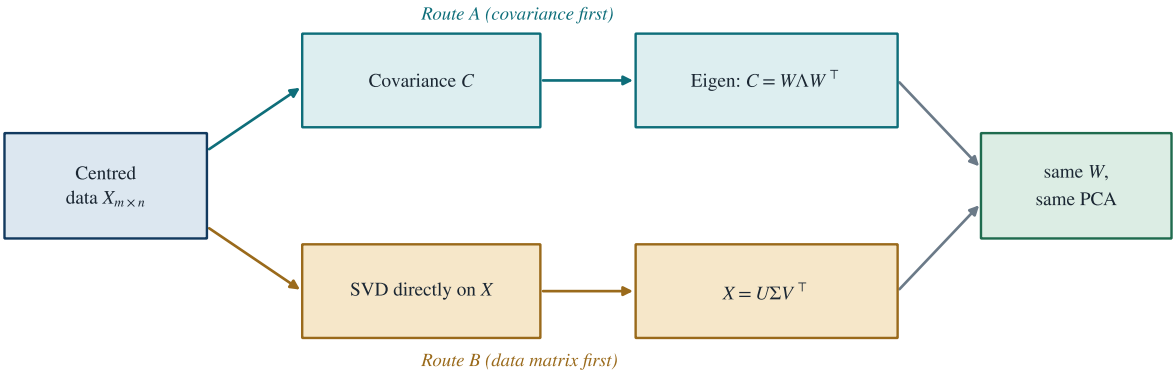


Fig. 2.1 Route A builds the covariance matrix first; Route B factorises the rectangular data matrix directly and reaches the same principal directions.

Route A — eigen-decomposition of the covariance matrix

$$C = \frac{1}{m-1} X^\top X \in \mathbb{R}^{n \times n}, \qquad C = W \Lambda W^\top \tag{2.1}$$

C is square and symmetric, so its eigenvectors are orthonormal and its eigenvalues are real and non-negative. Sorting gives $\lambda_1 \geq \lambda_2 \geq \ldots \geq \lambda_n \geq 0$; the corresponding eigenvectors w_i are the principal directions and the scores are $Z = XW_k$.

Route B — singular value decomposition of the data matrix

For **any** real matrix, square or rectangular, the SVD exists:

$$X = U \Sigma V^\top, \qquad U^\top U = I, V^\top V = I \tag{2.2}$$

Here $U \in \mathbb{R}^{m \times r}$ holds the left singular vectors (normalised scores), $\Sigma = \mathrm{diag}(\sigma_1, \ldots, \sigma_r)$ holds the singular values with $\sigma_1 \geq \sigma_2 \geq \ldots \geq 0$, and $V \in \mathbb{R}^{n \times r}$ holds the right singular vectors, which **are the principal directions** ($W = V$). The reduced representation is $Z = XV_k = U_k \Sigma_k$.

Derivation of the equivalence

Substituting the SVD into the covariance matrix gives

$$X^\top X = (U \Sigma V^\top)^\top (U \Sigma V^\top) = V \Sigma^\top U^\top U \Sigma V^\top = V \Sigma^2 V^\top \tag{2.3}$$

where $U^\top U = I$ removes the middle factor. Comparing with $C = W \Lambda W^\top$ term by term:

$$W = V \qquad \text{and} \qquad \Lambda = \frac{\Sigma^2}{m-1} \tag{2.4}$$

Hence the **eigenvalues are the scaled squares of the singular values**, the eigenvectors are the right singular vectors, and the ordering is identical because squaring preserves order. This is the exact statement that both routes give the same PCA.

$$\lambda_i = \frac{\sigma_i^2}{m-1}, \qquad \mathrm{EVR}_i = \frac{\sigma_i^2}{\sum_j \sigma_j^2} \tag{2.5}$$

Because the factor $1/(m-1)$ cancels in the ratio, the explained-variance percentages can be read straight from the squared singular values — the covariance matrix need never be formed.

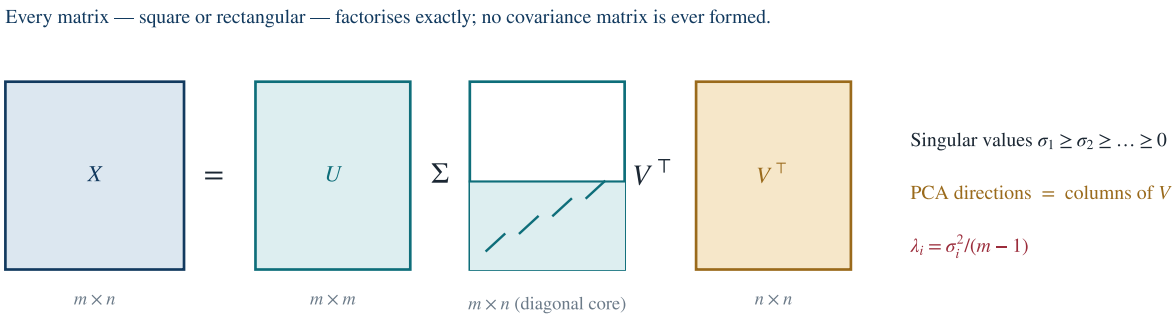


Fig. 2.2 Shapes in the full SVD of a rectangular matrix; only the first r columns matter, giving the economy (thin) SVD.

Comparison

Criterion	Eigen-decomposition	Singular value decomposition
Object factorised	$C = 1/(m-1)X^\top X$ (square, $n \times n$)	X itself ($m \times n$, any shape)
Applicability	Only square matrices — the covariance must be built first	Every real matrix, square or rectangular
Output	W (eigenvectors), Λ (variances)	U, Σ, V; $W = V$, $\lambda_i = \sigma_i^2/(m-1)$
Cost	$O(mn^2)$ to form C + $O(n^3)$ to factorise	$O(mn\min(m,n))$, no $n \times n$ matrix formed
Numerical accuracy	Squaring doubles the condition number: $\kappa(X^\top X) = \kappa(X)^2$	Works with $\kappa(X)$ — no squaring, hence more accurate
Catastrophic case	Fails when $m < n$: C is singular, rank $\le m$	Still exact; returns $\min(m,n)$ nonzero singular values
Sign convention	Signs of w_i arbitrary	Signs of u_i, v_i arbitrary (same ambiguity)
Practical use	Small n, or when C is already available	Default choice: NumPy *svd*, scikit-learn PCA, large or wide data

Why SVD is the more direct route for a rectangular matrix

A rectangular $X \in \mathbb{R}^{m \times n}$ has no eigenvectors — "eigen" is defined only for square operators. The eigen route therefore inserts an artificial intermediate step: it first squares the problem into $X^\top X$, factorises that, and then has to undo the scaling. SVD needs no such step; it factorises X exactly as given. Four concrete consequences follow.

1. **It always exists.** The SVD is defined for every matrix and always has $\min(m,n)$ terms, whereas $X^\top X$ is singular whenever $m < n$ (wide data, e.g. 100 samples of 10{,}000 genes) and half of the eigen-spectrum is then numerically meaningless zero.
2. **It is numerically better conditioned.** Forming $X^\top X$ squares the condition number, so directions with small singular values can disappear into round-off. SVD works on X directly and retains them.
3. **It is cheaper.** Forming C costs $O(mn^2)$ and factorising it $O(n^3)$; a thin SVD costs $O(mn\min(m,n))$. For wide matrices the difference is decisive.
4. **It gives the scores for free.** $Z = U_k\Sigma_k$ comes out of the same factorisation, and the discarded energy $\Sigma_{i>k}\sigma_i^2$ is read directly from Σ.

The eigen route keeps two advantages: it is easy to explain in an exam, and when the covariance matrix is already given (as in Question 1) it is the natural object to work on. It is also the route that reveals *why* PCA maximises variance, through $Cw = \lambda w$.

When to prefer which

- **Use SVD** when the input is a raw rectangular data matrix, when n is large, when $m < n$, or when numerical accuracy matters — this is the practical default.
- **Use eigen-decomposition** when the covariance matrix already exists, when n is small, or when the theory ($Cw = \lambda w$, Rayleigh quotient) must be demonstrated.

COMMON MISTAKES THAT LOSE MARKS

(i) Saying "SVD and eigen give different PCs" — they give the *same* directions. (ii) Forgetting the factor $m - 1$ when converting σ_i^2 into λ_i. (iii) Claiming eigenvectors exist for a rectangular matrix. (iv) Ignoring that both methods leave the **sign** of each component undetermined, so a comparison of components can look different after a sign flip.

GOLDEN LINE

Eigen route: covariance first, then factorise. SVD route: factorise the data directly. Right singular vectors of X are the eigenvectors of $X^\top X$ — for a rectangular matrix, SVD is the more direct and numerically safer route.

KEYWORDS TO WRITE IN THE EXAM

Eigen-decomposition · singular value decomposition · $X = U\Sigma V^\top$ · $X^\top X = V\Sigma^2 V^\top$ · $\lambda_i = \sigma_i^2/(m-1)$ · rectangular matrix · condition number · economy SVD · sign ambiguity

Q3 A deep-learning implementation produces unstable numerical values because exponentials become extremely large or small. Analyse overflow and underflow and propose mathematically sound numerical safeguards.

CO1 · L5 (Design) · 10 Marks

Where the problem comes from

Deep learning is dominated by two operations: **exponentials** (softmax, sigmoid, Gaussian likelihood) and **long products** (layer compositions, likelihoods over many samples). A computer stores real numbers in floating point, IEEE-754 float32, which has a finite window of representable magnitudes. Outside that window the hardware cannot represent the true value and the result is not an error — it is silently replaced by 0 or $\pm\infty$.

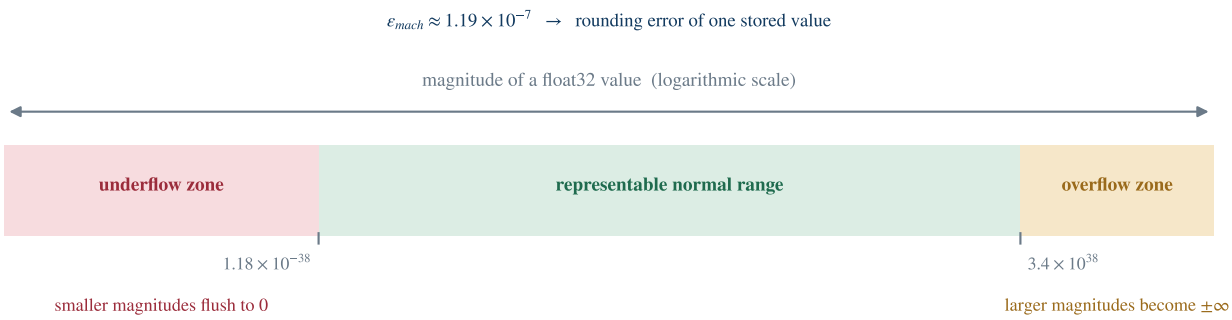


Fig. 3.1 Representable magnitudes of a float32 value: the underflow zone, the normal range and the overflow zone.

Key constants for float32: largest finite value $\approx 3.4 \times 10^{38}$, smallest normal positive $\approx 1.18 \times 10^{-38}$, machine epsilon $\approx 1.19 \times 10^{-7}$.

- **Overflow** occurs when a result exceeds the largest representable magnitude: $x > 3.4 \times 10^{38}$ becomes $+\infty$. Example: e^{1000} in a softmax, or a weight norm that explodes during training.
- **Underflow** occurs when a non-zero result becomes smaller than the smallest representable magnitude: $0 < |x| < 1.18 \times 10^{-38}$ is flushed to 0. Example: $\Pi_t p(y_t \mid y_{<t})$ over a 40-token sequence, or e^{-x} for large x.

- **Not-a-number (NaN)** is the secondary damage: $\infty - \infty$, $0 \cdot \infty$, ∞/∞, $0/0$ all produce NaN, and a single NaN propagates through every later gradient.

Why exponentials make this inevitable

For softmax with logits $z_1, \ldots, z_K$:

$$p_i = \frac{\exp(z_i)}{\sum_j \exp(z_j)} \tag{3.1}$$

If any z_i is large, $\exp(z_i)$ overflows to ∞ and the ratio becomes $\infty/\infty =$ NaN. If all z_i are very negative, every $\exp(z_i)$ underflows to 0 and the ratio becomes $0/0 =$ NaN. The same failure occurs in the sigmoid $\sigma(x) = \frac{1}{1+e^{-x}}$, where e^{-x} overflows for very negative x; and in the Gaussian log-likelihood.

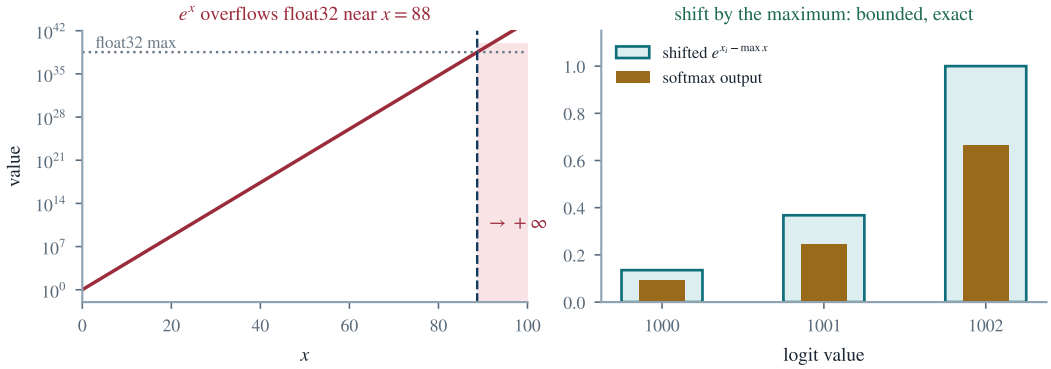


Fig. 3.2 Left: the exponential leaves the float32 range near $x = 88$. Right: shifting every logit by the maximum keeps all exponentials ≤ 1, so the softmax is computed exactly.

Safeguard 1 — the max-shift (log-sum-exp) trick

The softmax is **shift invariant**: multiplying numerator and denominator by e^{-a} changes nothing.

$$\frac{e^{z_i}}{\sum_j e^{z_j}} = \frac{e^{z_i - a}}{\sum_j e^{z_j - a}} \quad \text{for every } a \tag{3.2}$$

Choosing $a = \max_j z_j$ guarantees $z_i - a \leq 0$, so every exponential lies in $(0, 1]$: **overflow is impossible** and at least one term equals 1, so **total underflow is impossible**. Correctness is unchanged, only conditioning is improved.

$$\text{softmax}(z)_i = \frac{e^{z_i - \max_j z_j}}{\sum_j e^{z_j - \max_j z_j}}, \qquad \log \sum_j e^{z_j} = a + \log \sum_j e^{z_j - a} \tag{3.3}$$

The same identity gives the stable log-sum-exp, the workhorse for log-likelihoods and mixture models. In practice one never computes softmax and then a logarithm; one calls a single fused *log_softmax* primitive.

Safeguard 2 — work in the log domain

Products of probabilities become sums of log-probabilities, and sums do not overflow:

$$\log \prod_{t=1}^{T} p_t = \sum_{t=1}^{T} \log p_t \tag{3.4}$$

A likelihood of 10^{-300} underflows, but its log-likelihood -690.8 is an ordinary float. All objectives are therefore written as **negative log-likelihood**, and probabilities are exponentiated only at the final reporting step. Use *log1p(x)* instead of *log(1+x)* and *expm1(x)* instead of *exp(x)-1*, both of which keep full precision for small x.

Safeguard 3 — clip the arguments of exponentials and the gradients

$$\sigma(x) = \frac{1}{1+e^{-x}} (x \geq 0) \qquad \sigma(x) = \frac{e^{x}}{1+e^{x}} (x < 0) \tag{3.5}$$

In code: for $x \geq 0$ use $\sigma(x) = 1/(1 + e^{-x})$ (the exponential underflows harmlessly to 0, giving $\sigma = 1$); for $x < 0$ use $\sigma(x) = e^{x}/(1 + e^{x})$ (again a small, safe exponential). For the log-sigmoid use $-\text{softplus}(-x)$ with $\text{softplus}(u) = \log(1 + e^{u})$ computed stably, never $\log(\sigma(x))$ directly.

Gradient clipping caps the update so that a single explosive batch cannot destroy the weights:

$$g \leftarrow g \cdot \min\left(1, \frac{c}{\|g\|_2}\right) \tag{3.6}$$

This keeps the **direction** of the gradient and only rescales its magnitude, so learning continues instead of becoming NaN.

Safeguard 4 — condition the inputs and the parameters

- **Standardise inputs** ($x \leftarrow (x - \mu)/\sigma$) so that no layer sees values of magnitude 10^{6}.
- **Normalise activations** with batch/layer normalisation, which keeps pre-activations in a range where exponentials are safe.
- **Stable initialisation** (Xavier/He) keeps variances near 1 across layers, preventing products of matrices from exploding or vanishing.
- **Weight decay / regularisation** bounds $\|w\|$ so inner products $w^{\top} x$ cannot grow without limit.
- **Mixed precision with loss scaling:** compute in float16 but keep a master copy of weights in float32, and multiply the loss by a large scale factor so small gradients stay representable.

Safeguard 5 — avoid the numerically fragile formulations

Fragile expression	Why it fails	Numerically sound replacement
$\log(\text{softmax}(z))$	softmax underflows, $\log 0 = -\infty$	*log_softmax* fused with NLL
$w = (X^{\top}X)^{-1}X^{\top}y$	squaring doubles κ; singular $X^{\top}X$	QR or SVD: $w = V\Sigma^{-1}U^{\top}y$
$\sigma(x)$ then $\log \sigma(x)$	intermediate underflows to 0	$-\text{softplus}(-x)$
$\det(C)$ for log-likelihood	overflows/underflows for large n	Cholesky: $\log\det C = 2\Sigma_i \log L_{ii}$
$\Pi_i p_i$	underflows for long sequences	$\Sigma_i \log p_i$
$\exp(x) - \exp(y)$	cancellation for close values	stable form or *expm1*

Diagnostic procedure

1. Insert *assert torch.isfinite(loss).all()* after the loss; the first non-finite value localises the fault.
2. Check the loss at step 1: *inf* points to a softmax/likelihood, NaN points to a $0/0$ or $\infty - \infty$.
3. Log the maximum absolute value of each activation, gradient and weight. A growth of orders of magnitude per epoch is the signature of overflow.
4. Reduce the learning rate and enable gradient clipping to confirm that the cause is dynamics rather than a formula.
5. Once stable, re-introduce mixed precision with loss scaling.

HOW TO PHRASE THE ANSWER

Structure it as **cause** → **symptom** → **remedy**: overflow gives $\pm\infty$, underflow gives 0, both give NaN. Then give one mathematically justified safeguard per case (max-shift, log-domain, clipping) instead of merely saying "use float64".

GOLDEN LINE

Never exponentiate a large number and never multiply many small numbers. Subtract the maximum before exponentiating, add logarithms instead of multiplying probabilities, and clip the gradient norm.

KEYWORDS TO WRITE IN THE EXAM
Overflow · underflow · float32 range 3.4×10^{38} · machine epsilon · NaN · log-sum-exp trick · max-shift softmax · log-domain · log1p/expm1 · gradient clipping · stable sigmoid · log_softmax · loss scaling

Q4 Analyse how expectation, variance and covariance characterise a multivariate training dataset. Explain how the covariance structure motivates PCA.

CO1 · L4 (Analyse) · 10 Marks

Why these three quantities matter

A deep network sees only a cloud of m points in $\mathbb{R}^n$. Three statistics summarise that cloud completely for linear methods: **expectation** says where the cloud is centred, **variance** says how widely each coordinate spreads, and **covariance** says whether coordinates move together. Together they form the covariance matrix, which is the object PCA diagonalises.

Expectation

For a random variable X with density $p(x)$ and for a finite sample:

$$\mathbb{E}[X] = \int x\, p(x)\, dx \quad \approx \quad \hat{\mu} = \frac{1}{m}\sum_{i=1}^{m} x^{(i)} \tag{4.1}$$

For a vector $x \in \mathbb{R}^n$ the expectation is taken element-wise, $\mathbb{E}[x] = (\mathbb{E}[x_1], \ldots, \mathbb{E}[x_n])^\top$, and it is the **centre of mass** of the data. It is a linear operator:

$$\mathbb{E}[aX + bY] = a\,\mathbb{E}[X] + b\,\mathbb{E}[Y] \tag{4.2}$$

Variance

$$\mathrm{Var}(X) = \mathbb{E}\big[(X - \mathbb{E}[X])^2\big] = \mathbb{E}[X^2] - (\mathbb{E}[X])^2 \tag{4.3}$$

Variance is the **second central moment**; it measures average squared deviation from the mean and therefore the spread of one coordinate. Its square root σ is in the same units as the data, which is why standardisation divides by σ. The unbiased sample estimator uses $m - 1$:

$$\hat{\sigma}^2 = \frac{1}{m-1}\sum_{i=1}^{m} (x_i - \hat{\mu})^2 \tag{4.4}$$

For an affine transformation, $\mathrm{Var}(aX + b) = a^2\mathrm{Var}(X)$ — shifts do not change spread, and scaling scales variance by the square.

Covariance

$$\mathrm{Cov}(X, Y) = \mathbb{E}[(X - \mathbb{E}X)(Y - \mathbb{E}Y)] = \mathbb{E}[XY] - \mathbb{E}[X]\mathbb{E}[Y] \tag{4.5}$$

Covariance measures **joint** variation. Its sign is informative:

- $\mathrm{Cov} > 0$: both variables grow together (redundant information).
- $\mathrm{Cov} < 0$: one grows while the other falls (also redundant).
- $\mathrm{Cov} = 0$: no *linear* relationship (uncorrelated).

The correlation coefficient removes the units, so it is the interpretable scale-free version:

$$\rho_{XY} = \frac{\mathrm{Cov}(X, Y)}{\sigma_X \sigma_Y} \in [-1, 1] \tag{4.6}$$

The covariance matrix

Collecting all pairwise covariances of a centred data matrix into one array gives the **covariance (scatter) matrix**:

$$C = \frac{1}{m-1} X^\top X \in \mathbb{R}^{n \times n}, \qquad C_{jk} = \mathrm{Cov}(x_j, x_k) \tag{4.7}$$

- The **diagonal** $C_{jj} = \mathrm{Var}(x_j)$ gives the variance of each feature.
- The **off-diagonal** C_{jk} gives the redundancy between features j and k.
- C is symmetric and positive semi-definite, and its trace is the total variance: $\mathrm{trace}(C) = \Sigma_j \mathrm{Var}(x_j)$.

Under an affine map $z = Ax + b$ the covariance transforms as

$$\mathrm{Cov}(Az) = A\,\mathrm{Cov}(z)\,A^\top \tag{4.8}$$

This single identity is the engine of PCA: choosing $A = W^\top$ with W orthonormal gives $\mathrm{Cov}(W^\top x) = W^\top C W$, so PCA is the search for an A that makes the result diagonal.

How the covariance structure motivates PCA

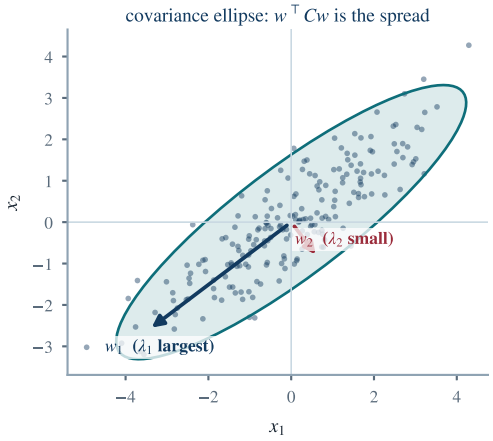


Fig. 4.1 The covariance ellipse of a 2-D cloud. Its axes are the eigenvectors of C, and their lengths are $\sqrt{\lambda_i}$; w_1 is the direction of maximum variance.

1. **Correlated features mean redundant coordinates.** If C has large off-diagonals, several columns of X carry nearly the same information. The data therefore occupies a thin, tilted region of $\mathbb{R}^n$, not a full box. Describing that region with its own axes needs fewer numbers than n.
2. **Variance is information.** A direction with near-zero variance is almost constant across samples and contributes almost nothing to distinguishing them. Dropping it loses almost nothing.
3. **The best axes are the eigen-directions of C.** For a unit direction w, the variance of the projection is the Rayleigh quotient $w^\top Cw$; maximising it gives $Cw = \lambda w$, i.e. eigenvectors, and the achieved variance is the eigenvalue.
4. **Diagonalisation removes redundancy.** With W the eigenvector matrix, $\mathrm{Cov}(W^\top x) = W^\top C W = \Lambda$ is diagonal: the new coordinates are uncorrelated. PCA is literally a change of basis into the eigenbasis of C.
5. **Total variance is conserved but redistributed.** Since $\mathrm{trace}(C) = \Sigma_i \lambda_i$, the rotation creates no new information; it concentrates the same total variance into the first few coordinates, which is exactly what makes truncation possible.

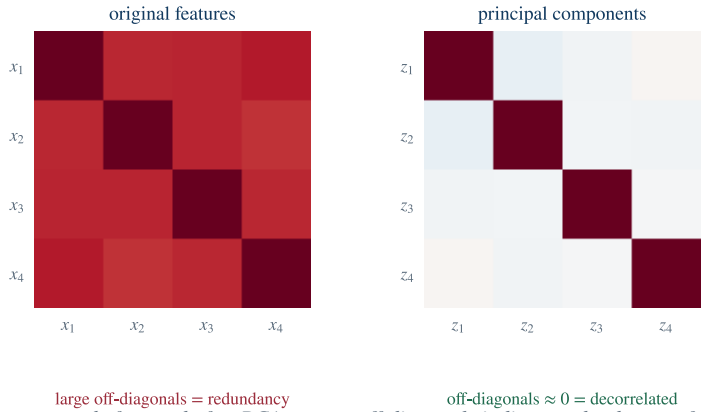


Fig. 4.2 Covariance (correlation) structure before and after PCA: strong off-diagonals indicate redundancy; after rotation the matrix is essentially diagonal.

Worked illustration

For the four-sample example of Question 1, C has diagonal 3.33 and off-diagonal 2.67. The diagonal entries say each feature has variance 3.33; the off-diagonal 2.67 is the same order of magnitude, so the two features are strongly redundant ($\rho = 0.8$). The eigenvalues 6 and 0.667 show why: one direction concentrates 90% of the total variance 6.67, the other only 10%. Averaging the two features along $w_1 = (1, 1)/\sqrt{2}$ *is* the first principal component.

Statistic	Formula	What it tells about the dataset	Role in PCA
Expectation	$\mu = 1/m \Sigma x^{(i)}$	location of the cloud	must be removed (centring)
Variance	$\mathrm{Var}(x_j) = \mathbb{E}[(x_j - \mu_j)^2]$	spread of one feature	becomes an eigenvalue λ_i
Covariance	$\mathrm{Cov}(x_j, x_k)$	linear redundancy between features	off-diagonal that PCA drives to zero
Covariance matrix	$C = 1/(m-1) X^\top X$	complete second-order description	factorised as $W \Lambda W^\top$
Trace	$\mathrm{trace}(C) = \Sigma_j \mathrm{Var}(x_j) = \Sigma_i \lambda_i$	total variance	denominator of the explained-variance ratio

Practical cautions

- Covariance captures only **linear** dependence: $\mathrm{Cov} = 0$ does not imply independence (e.g. $Y = X^2$).
- It is **not scale invariant**; features in different units must be standardised, which replaces C by the correlation matrix R.
- A single outlier can dominate C because deviations are squared; robust variants (trimmed or Huber covariance) exist.
- The sample covariance needs $m > n$ to be full rank; otherwise C is singular, which is another reason to prefer SVD.

LINK TO THE REST OF UNIT I

Covariance is the bridge between probability and linear algebra: Question 1 maximises $w^\top C w$, Question 2 factorises $X^\top X$, Question 7 solves $X^\top X w = X^\top y$, and Question 8 uses a diagonal covariance to speed up training. Writing that link explicitly earns analytical marks.

GOLDEN LINE

Expectation locates the cloud, variance measures its spread along one axis, and covariance measures how two axes move together. PCA rotates the axes so that the covariance matrix becomes diagonal, concentrating total variance into the fewest coordinates.

KEYWORDS TO WRITE IN THE EXAM

Expectation · variance · standard deviation · covariance · correlation · covariance matrix · positive semi-definite · trace = total variance · Rayleigh quotient · diagonalisation · decorrelation

Q5 **Derive the gradient-descent update for a general differentiable objective $J(\theta)$. Analyse how an excessively large and an excessively small learning rate affect convergence.**

CO1 · L4 (Analyse) · 10 Marks

The optimisation problem

Training a network means solving

$$\theta^* = \arg\min_{\theta \in \mathbb{R}^p} J(\theta) \tag{5.1}$$

where θ collects all weights and biases (p parameters). For a deep network J is non-convex and has no closed-form minimiser, so we iterate. Gradient descent is the simplest first-order iterative method: it uses only ∇J, never the Hessian.

First-order (Taylor) derivation

Expand the objective about the current point θ_k:

$$J(\theta_k + \Delta) \approx J(\theta_k) + \nabla J(\theta_k)^\top \Delta \tag{5.2}$$

We must choose a step Δ that decreases J, i.e. $\nabla J^\top \Delta < 0$. Among steps of fixed length $\|\Delta\| = \varepsilon$, the linear term is minimised when Δ is **antiparallel** to the gradient (Cauchy–Schwarz: $\nabla J^\top \Delta \geq -\|\nabla J\|\,\|\Delta\|$, with equality when $\Delta \propto -\nabla J$). Hence the direction of steepest descent is $-\nabla J$, and the update is

$$\theta_{k+1} = \theta_k - \eta \nabla J(\theta_k), \qquad \eta > 0 \tag{5.3}$$

This is gradient descent; η is the **learning rate** (step size). Written component-wise:

$$\theta_{k+1}^{(j)} = \theta_k^{(j)} - \eta \frac{\partial J}{\partial \theta^{(j)}}(\theta_k) \tag{5.4}$$

Working rule: if $\partial J/\partial \theta > 0$, increasing θ increases J, so θ moves down (subtract); if the derivative is negative, θ moves up. The minus sign enforces both cases.

Second-order justification and the bound on η

Assume J is L-smooth, i.e. $\|\nabla J(u) - \nabla J(v)\| \leq L\|u - v\|$, where $L = \lambda_{max}(\nabla^2 J)$ near a minimum. Then

$$J(\theta - \eta \nabla J) \leq J(\theta) - \eta\left(1 - \frac{\eta L}{2}\right)\|\nabla J\|^2 \tag{5.5}$$

The step **decreases** the objective whenever $0 < \eta < 2/L$. The curvature term $(1 - \eta L/2)$ is largest at $\eta = 1/L$, so $1/L$ is the theoretical optimal constant step. This is why the useful learning rate is set by the *curvature* of the loss, not by guessing.

Convergence on a quadratic

For the quadratic $J(\theta) = 1/2\theta^\top H\theta$ with H symmetric positive definite, the update is linear and can be solved exactly. Let $e_k = \theta_k - \theta^*$; then

$$e_{k+1} = e_k - \eta H e_k = (I - \eta H)e_k \Longrightarrow e_k = (I - \eta H)^k e_0 \tag{5.6}$$

In the eigenbasis of H this decouples into $e_i^{(k)} = (1 - \eta\lambda_i)^k e_i^{(0)}$. Convergence therefore requires

$$|1 - \eta\lambda_i| < 1 \quad \forall i \Leftrightarrow 0 < \eta < \frac{2}{\lambda_{max}} \tag{5.7}$$

and the slowest mode, controlled by the smallest eigenvalue, decays as $(1 - \eta\lambda_{min})^k$. This produces the two classical facts:

$$\text{condition number } \kappa = \frac{\lambda_{max}}{\lambda_{min}}, \qquad \text{iterations} \propto \kappa \log\frac{1}{\varepsilon} \tag{5.8}$$

An ill-conditioned problem (large κ) converges slowly even with the best η — the direct motivation for momentum, adaptive methods, and for PCA preprocessing (Question 8).

Effect of the learning rate

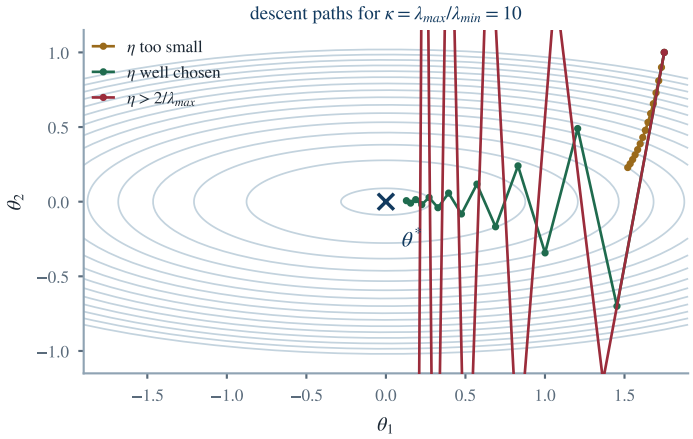


Fig. 5.1 Descent paths on a quadratic with condition number 10. A small rate crawls, a well-chosen rate converges steeply, and a rate above $2/\lambda_{max}$ oscillates and diverges.

- **η too small ($\eta \ll 1/L$).** Every step is safe, so J decreases monotonically, but each step makes only a tiny dent: convergence takes very many iterations, training appears "stuck", and finite iteration budgets end far from the minimum. Gradient noise is also under-damped relative to progress.
- **η too large ($\eta > 2/\lambda_{max}$).** The step overshoots the valley: J oscillates from wall to wall, then grows geometrically, and the parameters (and the loss) blow up to *inf*/NaN. In practice one sees a loss that jumps up and down or a sudden divergence after a few hundred steps.
- **η well chosen ($\eta \approx 1/L$).** Fast monotone decrease, no oscillation; this is the target, and it is usually found by a logarithmic search over $\{10^{-1}, 3\times10^{-2}, \ldots, 10^{-5}\}$.

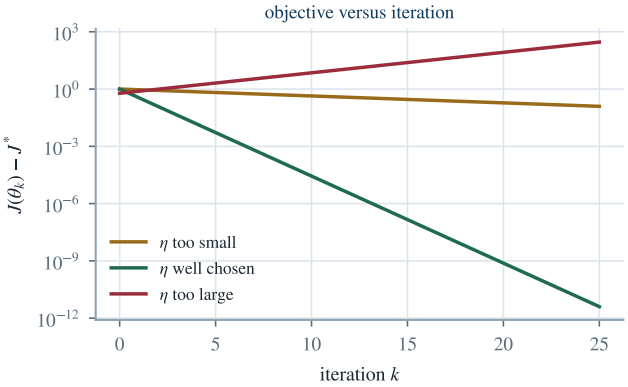


Fig. 5.2 Objective versus iteration for three learning rates on a log scale: too small (slow linear decay), well chosen (fast decay), too large (divergence).

Practical consequences of the analysis

1. **Scale the learning rate with curvature:** for batch size B, a common heuristic is $\eta \propto B$ (linear scaling) with a short warm-up.
2. **Decay η over time:** $\eta_t = \eta_0/(1 + t/T)$ or cosine decay, so that late training behaves like a small-step refinement.
3. **Precondition the problem:** normalising inputs, batch normalisation and PCA reduce κ, which effectively buys a larger safe η.
4. **Clip the gradient** so that an occasional huge ∇J cannot act like a temporary enormous learning rate.
5. **Monitor the loss curve:** a monotone but slow curve means η is small; a saw-tooth or divergent curve means η is large.

Algorithm

1. Initialise θ_0 (Xavier/He) and set η, tolerance ε, maximum iterations T.

2. Compute the full gradient $\nabla J(\theta_k) = 1/m \Sigma_i \nabla \ell(f(x^{(i)}; \theta_k), y^{(i)})$.
3. Update $\theta_{k+1} = \theta_k - \eta \nabla J(\theta_k)$.
4. Stop when $\|\nabla J(\theta_k)\| < \varepsilon$ or $k = T$; else repeat from step 2.

MARKS ARE LOST BY

Writing only "$\theta = \theta - \eta \nabla J$" with no derivation, ignoring the condition $0 < \eta < 2/\lambda_{max}$, and failing to distinguish *slow convergence* (small η) from *divergence* (large η). Show the Taylor expansion and the $(I - \eta H)$ recursion — those are the analytical parts of the question.

GOLDEN LINE

The update is $\theta \leftarrow \theta - \eta \nabla J(\theta)$ because $-\nabla J$ is the steepest-descent direction. Convergence needs $\eta < 2/\lambda_{max}$; too small is slow, too large diverges, and $\kappa = \lambda_{max}/\lambda_{min}$ decides the speed.

KEYWORDS TO WRITE IN THE EXAM

Gradient descent · steepest descent · Taylor expansion · learning rate · L-smoothness · step size $1/L$ · convergence condition · condition number · oscillation · divergence · learning-rate schedule

Q6 Formulate a constrained optimisation problem for minimising a model objective subject to a parameter-norm constraint. Explain the relationship between constrained optimisation and regularisation.

CO1 · L5 (Design) · 10 Marks

Formulation

Unconstrained training may choose parameters with an enormous norm, which fits noise and destroys generalisation. A natural cure is to **forbid** such parameters outright:

$$\min_{\theta} J(\theta) \quad \text{subjectto} \quad \|\theta\|_2 \le c, \qquad c > 0 \tag{6.1}$$

The feasible set $\mathcal{C} = \{\theta : \|\theta\|_2 \le c\}$ is a **closed convex ball**; J is the training loss (for example the mean squared error). Because the constraint set is convex and J is differentiable, a global minimum over $\mathcal{C}$ exists and the problem is well posed even when J is not convex globally.

Lagrangian and KKT conditions

Attach a multiplier $\lambda \ge 0$ to the constraint and write

$$\mathcal{L}(\theta, \lambda) = J(\theta) + \lambda\left(\|\theta\|_2^2 - c^2\right) \tag{6.2}$$

The Karush–Kuhn–Tucker conditions for this problem are

$$\nabla J(\theta^*) + 2\lambda\theta^* = 0, \qquad \lambda \ge 0, \qquad \lambda\left(\|\theta^*\|_2^2 - c^2\right) = 0 \tag{6.3}$$

The third condition (**complementary slackness**) splits the analysis into exactly two cases:

1. **Inactive constraint** ($\lambda = 0$): the unconstrained minimiser already satisfies $\|\theta^*\| < c$. Then $\nabla J(\theta^*) = 0$ and the constraint has no effect. This happens for small models or large c.
2. **Active constraint** ($\lambda > 0$): the unconstrained minimiser lies outside the ball, so the solution sits **on the boundary** $\|\theta^*\| = c$, with $\nabla J(\theta^*) = -2\lambda\theta^*$. At that point the gradient of the loss is *parallel* to the outward normal θ^* of the ball: the loss can no longer be reduced without leaving the allowed region.

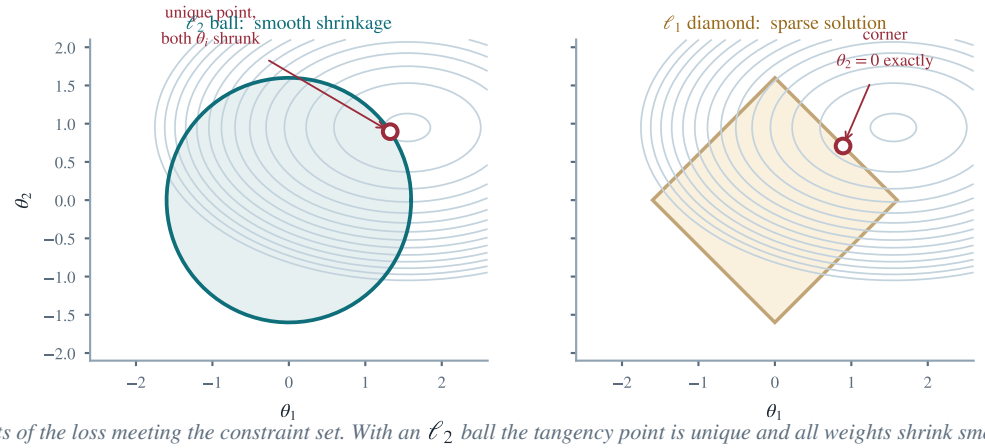


Fig. 6.1 Level sets of the loss meeting the constraint set. With an ℓ_2 ball the tangency point is unique and all weights shrink smoothly; with an ℓ_1 diamond the contact almost always occurs at a corner, so one weight becomes exactly zero.

Geometric interpretation

Draw the level curves $J(\theta) =$ const and grow the ball from the origin. The constrained optimum is the first point where a level curve touches the feasible set. Geometrically this is a **tangency condition**: the loss gradient is orthogonal to the constraint surface. That tangency is precisely what the KKT stationarity equation states.

From constraint to penalty (the equivalence)

Define the **penalised (regularised)** problem

$$\min_{\theta} J(\theta) + \lambda \|\theta\|_2^2 \tag{6.4}$$

Its stationarity condition is $\nabla J(\theta) + 2\lambda\theta = 0$ — **identical** to the KKT condition of the constrained problem with the same λ. Therefore, for every radius c there exists a multiplier $\lambda^*(c) \geq 0$ such that

$$\arg\min_{\|\theta\| \leq c} J(\theta) = \arg\min_{\theta} J(\theta) + \lambda^*(c)\|\theta\|_2^2 \tag{6.5}$$

and the map $c \mapsto \lambda^*$ is monotonically decreasing: a tighter constraint (smaller c) needs a larger penalty (larger λ). Conversely, for every λ the penalised solution lies on the boundary of some ball of radius $c(\lambda) = \|\theta_\lambda^*\|$. This is the **Lagrangian duality** that justifies ridge regression and weight decay: the constrained and the penalised formulations describe the same family of solutions.

$$\text{tighter ball } c \downarrow \quad \Longleftrightarrow \quad \text{stronger penalty } \lambda \uparrow \tag{6.6}$$

Two norms, two kinds of shrinkage

Aspect	ℓ_2 constraint / ridge penalty	ℓ_1 constraint / lasso penalty
Feasible set	sphere / ball (smooth)	diamond / cross-polytope (corners)
Solution location	tangency at a smooth point — unique and stable	usually a corner: some $\theta_j = 0$ exactly
Effect on weights	shrinks all weights smoothly, none becomes zero	performs variable selection (sparse model)
Gradient	differentiable everywhere ($2\lambda\theta$)	non-differentiable at 0 (needs subgradients / soft thresholding)
Closed form (linear model)	$w = (X^\top X + \lambda I)^{-1} X^\top y$	no closed form; $\text{sign}(w)\max(\lvert w\rvert - \lambda, 0)$ after orthonormalisation
Typical use	default regulariser, stabilises ill-conditioned problems	interpretable, compressed models

Ridge has a revealing spectral form. If $X^\top X = Q\Lambda Q^\top$, then

$$w_{\text{ridge}} = Q\,\text{diag}\left(\frac{\lambda_i}{\lambda_i + \alpha}\right) Q^{\top} w_{\text{OLS}} \tag{6.7}$$

so each eigen-direction is shrunk by the factor $\lambda_i/(\lambda_i + \alpha)$: directions with small variance (small λ_i) are shrunk most, which is exactly why ridge stabilises ill-conditioned problems and why $\alpha > 0$ always makes $X^{\top}X + \alpha I$ invertible.

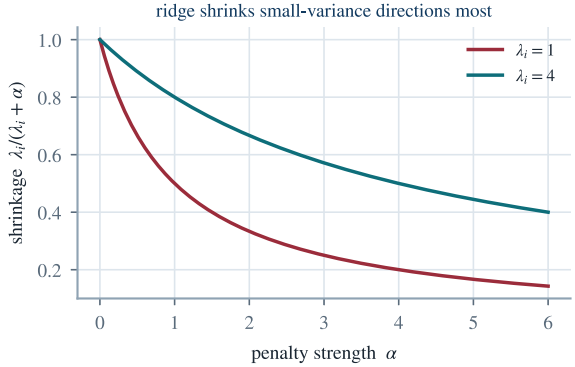


Fig. 6.2 Ridge shrinkage factor per eigen-direction. Small-variance directions are suppressed most, so the effective capacity of the model falls as the penalty grows.

Why this improves generalisation

- **Restricting capacity.** Limiting $\|\theta\|$ limits the complexity of the representable functions, which lowers variance at the cost of a small increase in bias — the bias–variance trade-off performed on purpose.
- **Suppressing noise directions.** Directions with little data support get large variance in the unconstrained solution; shrinkage removes them.
- **Numerical stability.** αI regularises the spectrum of $X^{\top}X$, so the solution no longer amplifies small singular values.
- **Implicit equivalence.** Early stopping, small initialisation and dropout are also forms of norm restriction: they keep the parameters near a small-norm region, which is why they behave like regularisers.

Design procedure

1. Standardise features, so the norm constraint is not dominated by units.
2. Choose the regulariser family: ℓ_2 (default), ℓ_1 (sparsity) or elastic net ($\ell_1 + \ell_2$).
3. Select λ (equivalently c) **on validation data**, searching logarithmically, e.g. $\lambda \in \{10^{-4}, \ldots, 10^{1}\}$.
4. Train with the penalty inside the objective — never by clipping weights after the update, which changes the descent dynamics.
5. Verify that the training and validation curves move together; a large gap still remaining means λ is too small.

ONE LINE THAT EARNS THE MARKS

"The constrained problem $\min J(\theta)$ s.t. $\|\theta\| \le c$ has KKT condition $\nabla J + 2\lambda\theta = 0$, which is the same as the stationarity condition of the penalty problem $\min J(\theta) + \lambda\|\theta\|^2$; hence a norm constraint and a norm penalty are two views of the same restriction, with c and λ inversely related."

GOLDEN LINE

A norm constraint caps model capacity; its KKT multiplier becomes the penalty weight. ℓ_2 shrinks smoothly, ℓ_1 also sparsifies, and both are the same idea as seen from the constraint side and the penalty side.

KEYWORDS TO WRITE IN THE EXAM

Constrained optimisation · feasible set · Lagrangian · KKT conditions · complementary slackness · active/inactive constraint · norm constraint · regularisation · ridge · lasso · sparsity · weight decay · duality

Q7 **For a least-squares problem $J(w) = \|Xw - y\|^2$, derive the normal equations and discuss the conditions under which the solution is unique.**

CO1 · L4 (Analyse) · 10 Marks

Problem statement

Given a design matrix $X \in \mathbb{R}^{m \times n}$ (m samples, n features) and targets $y \in \mathbb{R}^m$, find the weight vector that minimises the sum of squared errors:

$$J(w) = \|Xw - y\|_2^2 = (Xw - y)^\top (Xw - y) = \sum_{i=1}^{m} \left(x^{(i)\top} w - y^{(i)}\right)^2 \tag{7.1}$$

This is the loss of a **linear model with a squared-error criterion**, and it is also the negative log-likelihood of a Gaussian noise model (Question 5 of Unit II).

Derivation of the normal equations

Expand the objective:

$$J(w) = w^\top X^\top X w - 2 w^\top X^\top y + y^\top y \tag{7.2}$$

Differentiate with respect to the vector w (using $\frac{\partial a^\top w}{\partial w} = a$ and $\frac{\partial w^\top A w}{\partial w} = 2Aw$ for symmetric A):

$$\frac{\partial J}{\partial w} = 2X^\top X w - 2X^\top y \tag{7.3}$$

Setting the gradient to zero gives the **normal equations**:

$$X^\top X w = X^\top y \qquad \Longrightarrow \qquad \hat{w} = (X^\top X)^{-1} X^\top y \tag{7.4}$$

(Second-order check: $\nabla^2 J = 2X^\top X$ is positive semi-definite, so any stationary point is a global minimum — J is convex.)

Geometric derivation (orthogonality principle)

Xw is a linear combination of the columns of X, so it ranges over the **column space** $\mathrm{col}(X)$. Minimising $\|Xw - y\|$ means finding the point of that subspace closest to y — the orthogonal projection. At the optimum the residual must be perpendicular to every column of X:

$$X^\top (y - X\hat{w}) = 0 \Leftrightarrow X^\top X \hat{w} = X^\top y \tag{7.5}$$

Both derivations give the same equations; the geometric one explains *why* they are correct.

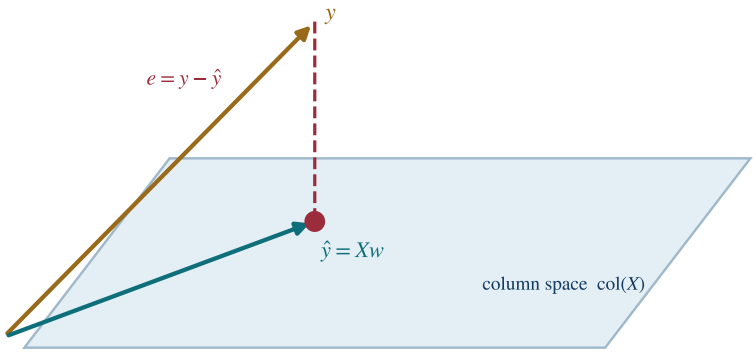


Fig. 7.1 Least squares as an orthogonal projection: $\hat{y} = Xw$ is the closest point of col(X) to y, and the residual $e = y - \hat{y}$ is orthogonal to the whole subspace.

The projection (hat) matrix

$$\hat{y} = X(X^\top X)^{-1}X^\top y = Hy, \qquad H = X(X^\top X)^{-1}X^\top \tag{7.6}$$

H is symmetric ($H^\top = H$) and **idempotent** ($H^2 = H$): projecting twice changes nothing. Its trace equals n, the dimension of the subspace, and $I - H$ projects onto the residual space. The minimum value of the objective is $\|(I - H)y\|^2$.

Uniqueness of the solution

The solution exists and is **unique if and only if** $X^\top X$ **is invertible**, which for an $m \times n$ matrix is equivalent to the following conditions:

$$\text{rank}(X) = n \Leftrightarrow \text{columns of } X \text{ are linearly independent} \Leftrightarrow \text{null}(X) = \{0\} \tag{7.7}$$

Equivalently, all n singular values of X are non-zero, so $\kappa(X) < \infty$. This requires $m \geq n$ (at least as many samples as features) *and* no redundancy among the features.

Situation	Rank / shape	Consequence for $X^\top X$	Nature of the solution
Well-posed (tall, independent)	$m \geq n$, rank n	invertible, κ finite	unique $\hat{w} = (X^\top X)^{-1}X^\top y$
Under-determined (wide)	$m < n$	singular, rank $\leq m < n$	infinitely many exact solutions ($J = 0$)
Collinear / duplicate features	rank $< n$	singular	infinitely many solutions, same $\hat{y}$
Constant column + intercept	rank deficiency	singular	intercept and slope not identifiable separately
Ridge-regularised	any shape	$X^\top X + \alpha I \succ 0$	always unique: $(X^\top X + \alpha I)^{-1}X^\top y$

When the solution is not unique, the set of minimisers is an affine subspace, and the standard choice is the **minimum-norm** solution obtained from the Moore–Penrose pseudo-inverse:

$$\hat{w} = X^+ y = V\Sigma^{-1}U^\top y, \qquad \hat{w} = \arg\min\{\|w\|_2 : Xw = \hat{y}\} \tag{7.8}$$

Note that $\hat{y} = X\hat{w}$ remains unique even when $\hat{w}$ does not: the projection onto a subspace is always unique.

Numerical considerations (analysis marks)

1. **Never invert $X^\top X$ directly.** Forming $X^\top X$ squares the condition number, $\kappa(X^\top X) = \kappa(X)^2$, so half of the significant digits can be lost.
2. **Use QR:** factor $X = QR$ with $Q^\top Q = I$ and R upper triangular; then $X^\top X = R^\top R$ and the normal equations reduce to the triangular system $R\hat{w} = Q^\top y$, solved by back substitution in $O(n^2)$ with condition number $\kappa(R) = \kappa(X)$.
3. **Use SVD for rank-deficient or ill-conditioned data:** the pseudo-inverse truncates negligible singular values, giving a stable minimum-norm answer.
4. **Add ridge** ($\alpha > 0$) when features are collinear; this guarantees uniqueness and bounds the solution norm.

Worked example

Fit $y = w_0 + w_1 x$ to the three points $(0, 1), (1, 2), (2, 4)$, so that X has columns $(1, 1, 1)^\top$ and $(0, 1, 2)^\top$ and $y = (1, 2, 4)^\top$.

$$X^\top X = \begin{bmatrix} 3 & 3 \\ 3 & 5 \end{bmatrix}, \qquad X^\top y = \begin{bmatrix} 7 \\ 11 \end{bmatrix} \tag{7.9}$$

Solving the two equations in w gives $w_1 = 1.5$, $w_0 = 0.833$, so the fitted line is $\hat{y} = 0.833 + 1.5x$ and the residual sum of squares is 0.167. The matrix $X^\top X$ has determinant $3 \cdot 5 - 9 = 6 \neq 0$, so the fit is unique — the two columns are not collinear.

EXAMINER'S CHECKLIST

(i) Expand J, (ii) differentiate to $2X^\top Xw - 2X^\top y$, (iii) state $X^\top Xw = X^\top y$, (iv) give $\hat{w} = (X^\top X)^{-1}X^\top y$, (v) prove minimality via positive semi-definiteness **or** the orthogonality principle, (vi) state the uniqueness condition $\text{rank}(X) = n$, and (vii) mention QR/SVD for numerical stability.

GOLDEN LINE

Least squares is an orthogonal projection of y onto $\text{col}(X)$; setting the residual orthogonal to that subspace gives the normal equations $X^\top Xw = X^\top y$. The solution is unique exactly when X has full column rank.

KEYWORDS TO WRITE IN THE EXAM

Least squares · normal equations · residual · orthogonal projection · hat matrix · idempotent · full column rank · uniqueness · under-determined · pseudo-inverse · QR decomposition · condition number · ridge

Q8 **A feature matrix contains highly correlated variables. Evaluate whether PCA is appropriate for preprocessing before a neural network, and justify the decision using variance and decorrelation.**

CO1 · L5 (Design) · 10 Marks

The problem with correlated inputs

Suppose the input matrix has features that are almost linear combinations of each other, so the covariance matrix C has large off-diagonal entries and several small eigenvalues. For a network this causes three concrete problems.

1. **Redundant parameters.** The first layer spends capacity learning to disentangle directions that carry the same information.
2. **Ill-conditioned optimisation.** The Hessian of the loss inherits the spectrum of C. With $\kappa = \lambda_{max}/\lambda_{min} \gg 1$, gradient descent needs about $\kappa \log(1/\varepsilon)$ iterations (Question 5): the loss surface is a narrow valley and training is slow.
3. **Unstable weights.** Collinear inputs make the optimal weights non-unique; different runs give very different but equally accurate solutions, and the weights change violently under small perturbations of the data.

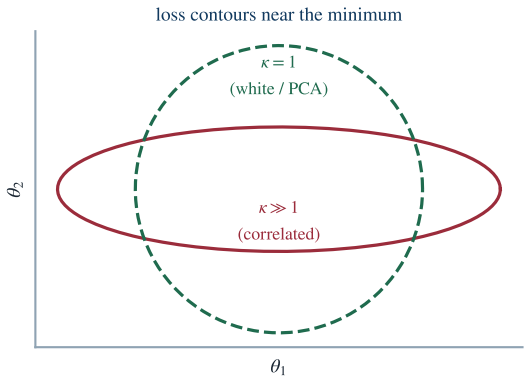


Fig. 8.1 Covariance (hence loss) contours of a correlated problem versus a decorrelated one. Round contours mean every direction has similar curvature, so one learning rate suits all parameters.

What PCA does to the inputs

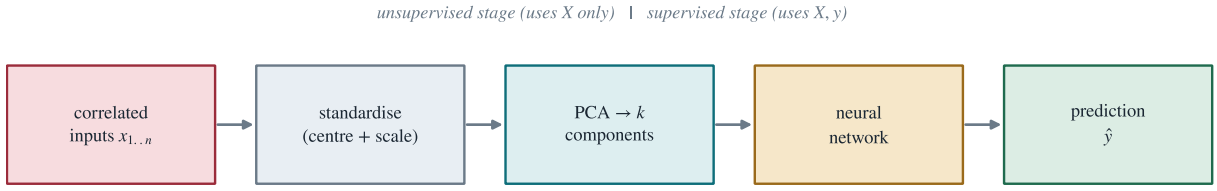


Fig. 8.2 Where PCA sits in the pipeline: an unsupervised reduction stage followed by the supervised network.

1. Centre the training data and compute $C = 1/(m-1)X^\top X$, with eigenvalues $\lambda_1 \geq \ldots \geq \lambda_n$.
2. Factorise $C = W\Lambda W^\top$ and keep the top k eigenvectors W_k.
3. Project $Z = XW_k$. Then $\mathrm{Cov}(Z) = W_k^\top CW_k = \Lambda_k$ is **diagonal**: the new inputs are uncorrelated, with variances $\lambda_1, \ldots, \lambda_k$.
4. Optionally **whiten** so that all directions have unit variance: $Z_{\text{white}} = XW_k\Lambda_k^{-1/2}$, giving $\mathrm{Cov}(Z_{\text{white}}) = I$ and $\kappa = 1$.

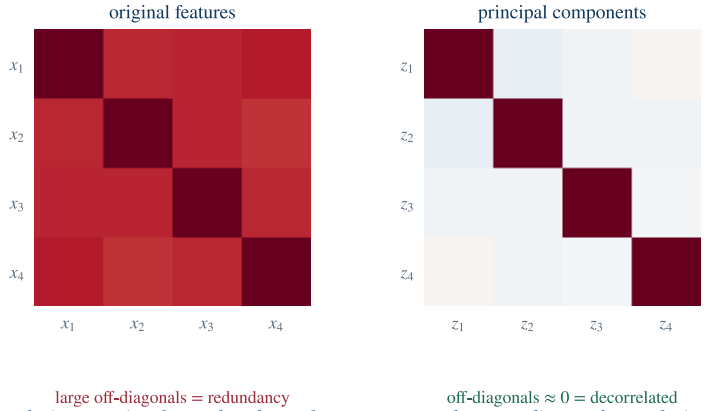


Fig. 8.3 Before and after: the correlation matrix of correlated raw features versus the near-diagonal correlation matrix of principal components.

Benefits for the network

Effect	Mechanism	Practical consequence
Decorrelation	$\mathrm{Cov}(Z) = \Lambda_k$ diagonal	round loss contours, one learning rate suits all weights
Better conditioning	κ falls from $\lambda_{max}/\lambda_{min}$ to λ_{max}/λ_k (to 1 after whitening)	fewer iterations to reach the same loss
Fewer inputs	$n \to k$	first-layer parameters fall from $(n+1)h$ to $(k+1)h$; less memory and compute
Noise removal	low-variance directions discarded	small random fluctuations are suppressed, which can improve generalisation
Redundancy removal	components are orthogonal	no duplicated information to disentangle
Visualisation	scores z_1, z_2	data can be plotted and outliers inspected

The parameter saving is easy to quantify: a first hidden layer of $h = 128$ units on $n = 256$ correlated inputs needs $257 \times 128 = 32{,}896$ weights; after PCA to $k = 40$ it needs $41 \times 128 = 5{,}248$ — an 84% reduction with almost no variance lost.

The risks — when PCA is the wrong choice

These determine whether the answer is "yes" or "no", and they carry the evaluation marks.

1. **Variance is not relevance.** PCA is *unsupervised*: it keeps the directions with the largest variance, which need not be the directions that predict y. A low-variance feature that is highly predictive can be thrown away. This is a real loss of accuracy, not a theoretical worry.

2. **Linearity.** PCA only removes *linear* correlation. If the informative structure is nonlinear, an autoencoder or kernel PCA is needed.
3. **Scale dependence.** The variance ordering is meaningless if features have different units, so standardisation must precede PCA.
4. **Loss of interpretability.** Each component is a mixture of all original features, so a model built on components cannot be explained feature-by-feature (a problem in clinical or credit-risk settings).
5. **Leakage.** The mean, scaling and W_k must be fitted on the **training split only** and then applied to validation and test data; otherwise the test information leaks into training.
6. **Deeper networks already adapt.** A stacked network can learn its own linear mixture in the first layer, so the benefit is largest for shallow or small models and for classical classifiers (SVM, logistic regression) attached to the features.

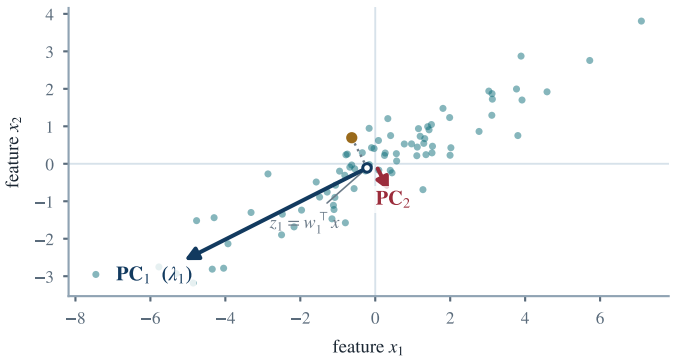


Fig. 8.4 What is kept and what is lost: the spread along w_1 is preserved, while the (small) spread along w_2 is discarded — correct only if the discarded direction is irrelevant.

Decision procedure

1. Standardise the features and inspect the correlation matrix C (or R). Off-diagonals near ± 1 confirm strong linear redundancy.
2. Check the spectrum: if one or two eigenvalues dominate (e.g. $\text{Cum}_2 > 0.95$), the data is intrinsically low-dimensional and PCA is appropriate.
3. Choose k by **cross-validation of the final network accuracy**, not by variance alone — and compare against the un-reduced baseline.
4. Fit W_k on the training fold only; apply the same transformation to validation and test data. Re-fit inside every cross-validation fold.
5. Compare three pipelines: (a) no PCA, (b) PCA with 95% variance, (c) PCA with whitening. Keep the one with the best validation score at comparable cost.
6. If the discarded low-variance directions turn out to be predictive (accuracy drops), abandon PCA and use feature selection, an autoencoder, or a network with more capacity instead.

Verdict

For a feature matrix with **high linear correlation** ($|\rho| > 0.9$), a **skewed spectrum** and a **shallow or classical model**, PCA is appropriate: it decorrelates the inputs, improves conditioning, saves parameters and can denoise. For **nonlinear** structure, for problems where **every direction may matter** (small k, weak signal), or where **interpretability is required**, PCA should not be applied blindly; an autoencoder or feature selection is preferable. The defensible answer is therefore conditional, and must be justified by the covariance spectrum and confirmed by validation accuracy.

MAKE THE EVALUATION EXPLICIT

The question asks you to *evaluate*, so a one-sided "PCA is always good" answer caps the marks. Give the spectrum argument for, the "variance $\neq$ relevance" argument against, and close with the cross-validated decision rule.

GOLDEN LINE

PCA before a network buys decorrelation, better conditioning and fewer parameters, at the risk of discarding low-variance but predictive directions. Decide from the covariance spectrum and confirm by validation accuracy.

KEYWORDS TO WRITE IN THE EXAM

PCA preprocessing · correlated features · decorrelation · covariance spectrum · conditioning · whitening · parameter reduction · variance $\neq$ relevance · data leakage · autoencoder alternative

Rapid revision — the whole unit in one grid

Read this column-wise the night before the examination: the trigger phrase, the three things that must appear, and the trap that costs marks.

Question	Three things that must appear	The trap
1 PCA	centring; $C = W\Lambda W^\top$; eigen $=$ variance; $Z = XW_k$; cumulative-variance rule	forgetting to centre the data
2 Eigen vs SVD	$X = U\Sigma V^\top$; $X^\top X = V\Sigma^2 V^\top$; $\lambda_i = \sigma_i^2/(m-1)$	saying the two routes give different PCs
3 Overflow/underflow	float32 limits; max-shift; log-domain; gradient clipping; NaN sources	"just use float64" without a mathematical fix
4 Moments	$\mathbb{E}$, Var, Cov; covariance matrix; trace $=$ total variance; $Cw = \lambda w$	claiming $\mathrm{Cov} = 0$ implies independence
5 Gradient descent	Taylor/steepest-descent derivation; $0 < \eta < 2/\lambda_{max}$; $(I - \eta H)^k$ recursion	no derivation, only the update rule
6 Constrained optimisation	Lagrangian, KKT, complementary slackness; $\nabla J + 2\lambda\theta = 0$; constraint $\Leftrightarrow$ penalty	confusing ℓ_1 and ℓ_2 geometry
7 Least squares	expansion $\rightarrow$ gradient; $X^\top Xw = X^\top y$; orthogonality; rank condition	inverting $X^\top X$ when it is singular
8 PCA before a network	decorrelation, conditioning, parameter saving; variance $\neq$ relevance; CV decision	one-sided "PCA is always good"